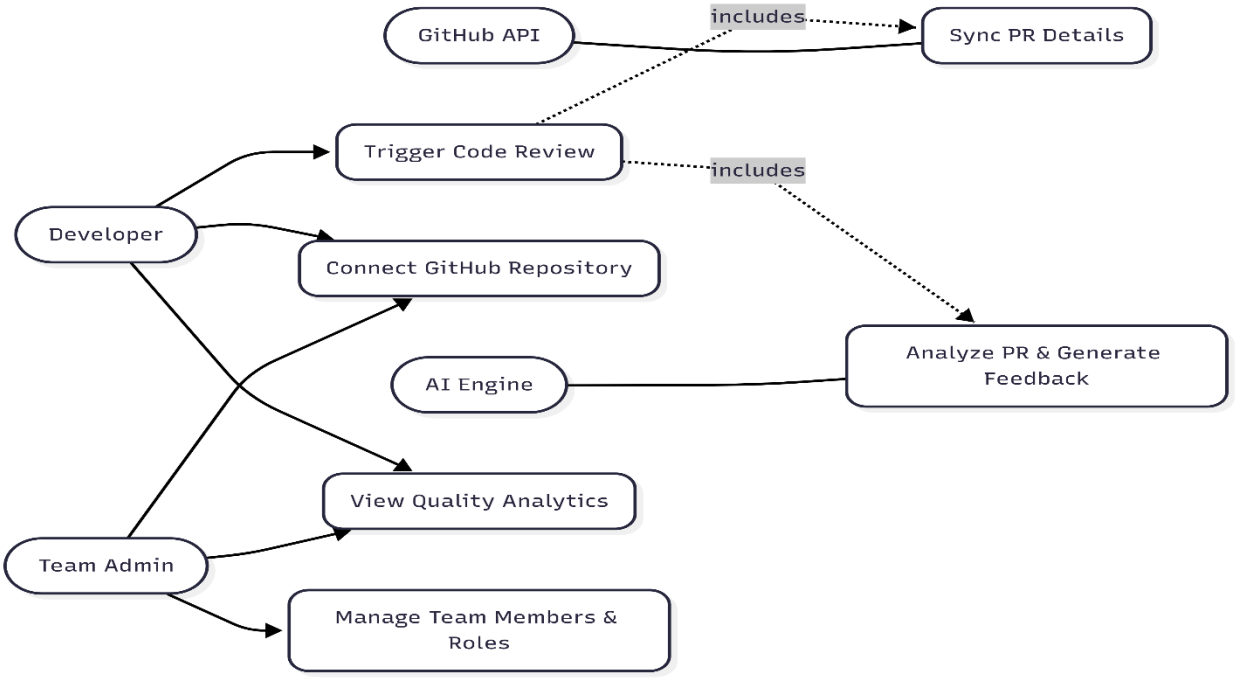
DevReview AI: System Analysis & Design

1-system Analysis & Objectives

1.1-Problem & Goals

In modern software development, manual code reviews are time-consuming, prone to human error, and often fail to detect complex security vulnerabilities or maintainability issues before code is merged. Development teams struggle to maintain consistent code quality standards while keeping up with agile delivery cycles, leading to technical debt and potential production failures. Automate Reviews: Instant AI-powered analysis for GitHub PRs to detect bugs and code smells.

- **Automate Code Reviews:** Provide instant, AI-driven code analysis for GitHub Pull Requests to detect bugs, security flaws, and code smells.
- **Enhance Team Collaboration:** Enable real-time, interactive comment threads on code reviews allowing developers to resolve issues collaboratively.
- **Improve Security & Compliance:** Automatically enforce branch protection rules and scan for vulnerabilities using advanced AI models (Gemini, Groq, etc.).
- **Provide Actionable Insights:** Generate detailed analytics and risk scores for repositories to help tech leads track code quality trends over time.



Functional Requirements:

1. **Authentication:** The system must allow users to sign up and log in using GitHub OAuth or Email/Password via Better-Auth.

2. **GitHub Integration:** The system must automatically fetch PR details, code diffs, and post review summaries directly to GitHub as comments.
3. **AI Analysis:** The system must support multiple AI providers (Google Gemini, Groq) to analyze code based on user-defined severity rules.
4. **Real-time Collaboration:** The system must update review threads and comment statuses in real-time without page reloads using Pusher WebSockets.
5. **Background Processing:** Long-running AI reviews must be processed asynchronously using Inngest to prevent frontend timeouts.

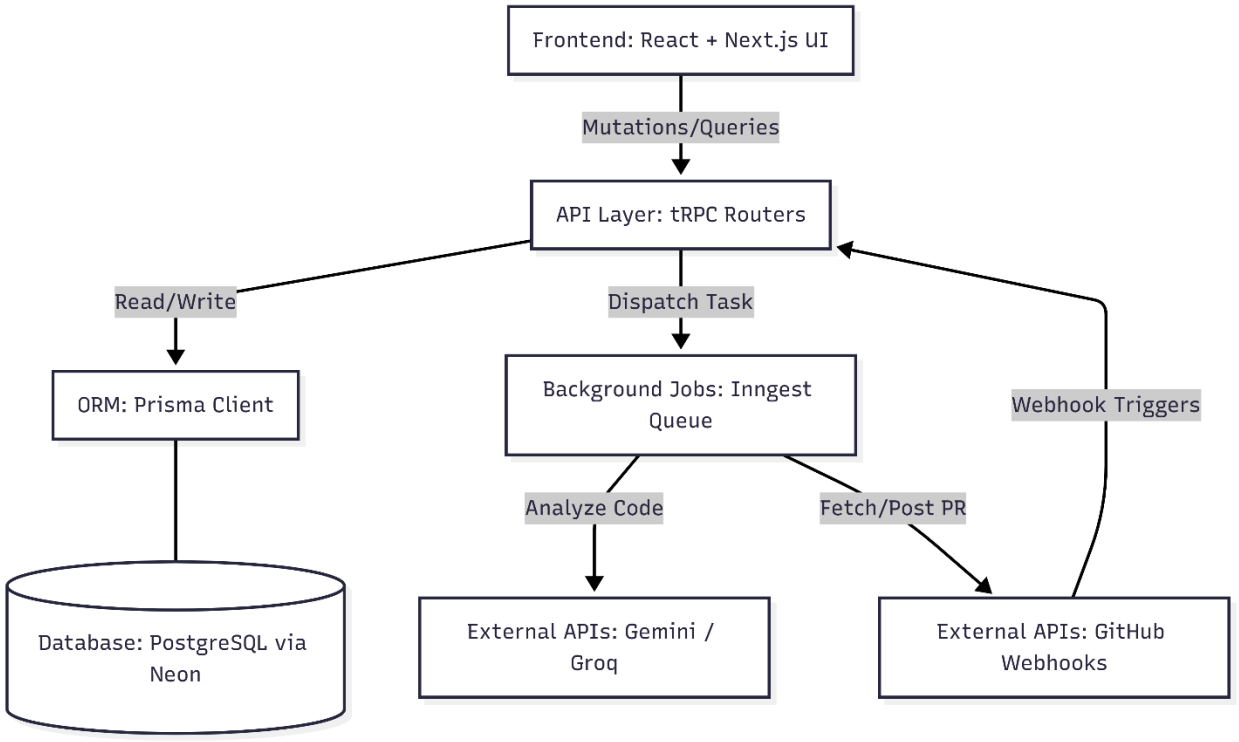
Non-Functional Requirements:

1. **Performance:** The frontend dashboard must load within 2 seconds, and real-time updates must reflect in less than 500ms.
2. **Security:** API endpoints must be protected using Upstash Redis Rate Limiting. Sensitive tokens (like GitHub Access Tokens) must be encrypted.
3. **Scalability:** The application must utilize a Serverless architecture (Next.js APIs + Serverless PostgreSQL on Neon) to handle concurrent webhook payloads efficiently.
4. **Availability:** The system should maintain 99.9% uptime, utilizing fallback mechanisms if a specific AI provider's API goes down.

1.2**-Software Architecture**

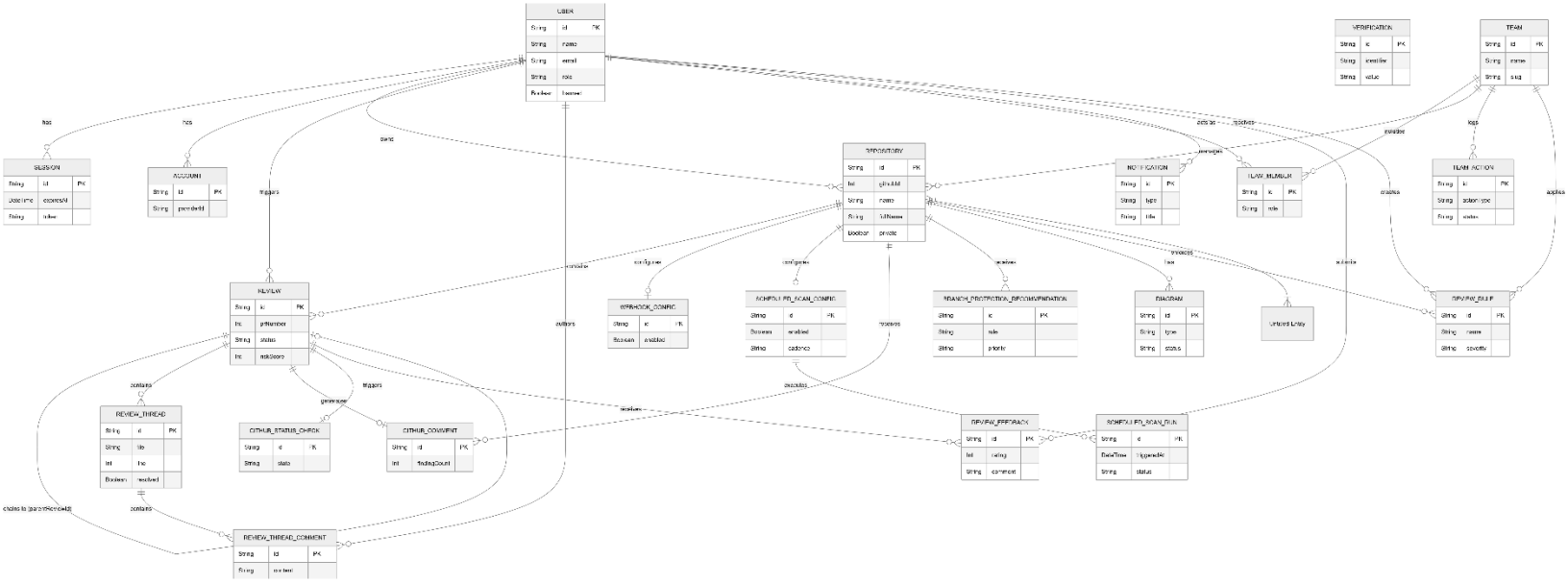
The system adopts a **Modern Serverless Monorepo Architecture**, heavily utilizing the **Client-Server** model tailored for Next.js 16. It replaces traditional REST APIs with **tRPC** for end-to-end type safety between the frontend and backend.

- **Frontend Layer (Client):** Built with React 19 and Next.js App Router. It handles state management, routing, and real-time UI updates (using GSAP and TailwindCSS).
- **API Layer (Server):** Next.js API routes host the tRPC routers, which act as the controllers processing business logic and validation (using Zod).
- **Data Access Layer:** Prisma ORM acts as the bridge connecting the backend logic to the PostgreSQL database, ensuring safe and structured database queries.
- **Asynchronous Workers:** Inngest is used to decouple heavy tasks (like AI processing and GitHub API fetching) from the main request-response cycle, operating as a background job queue.



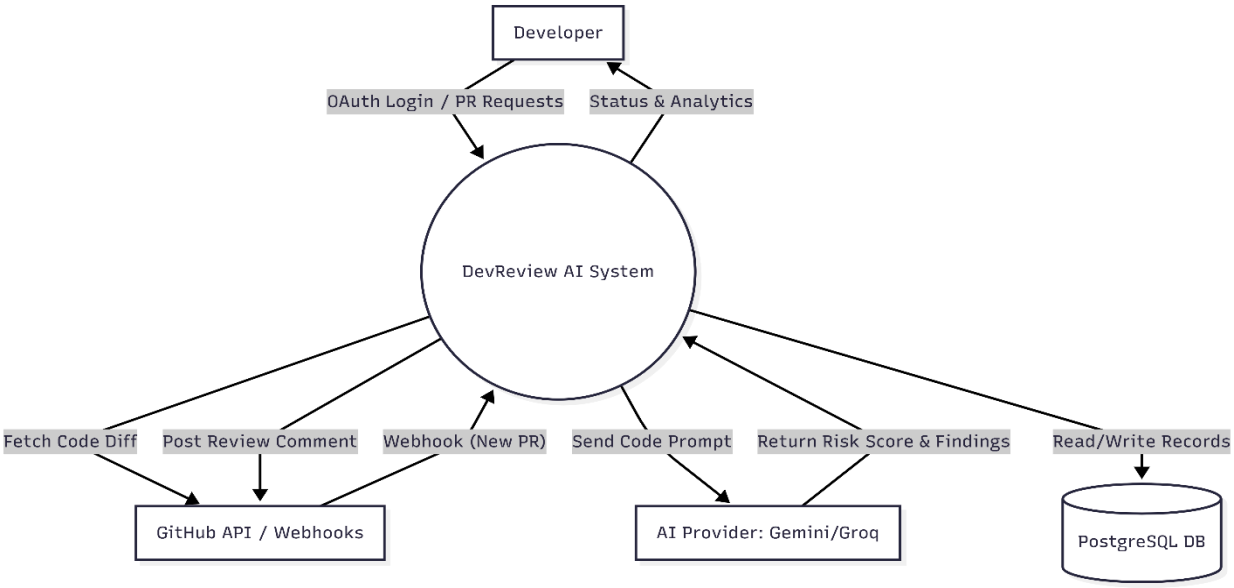
2. Database Design & Data Modeling

2.1 ER Diagram (Entity-Relationship Diagram)

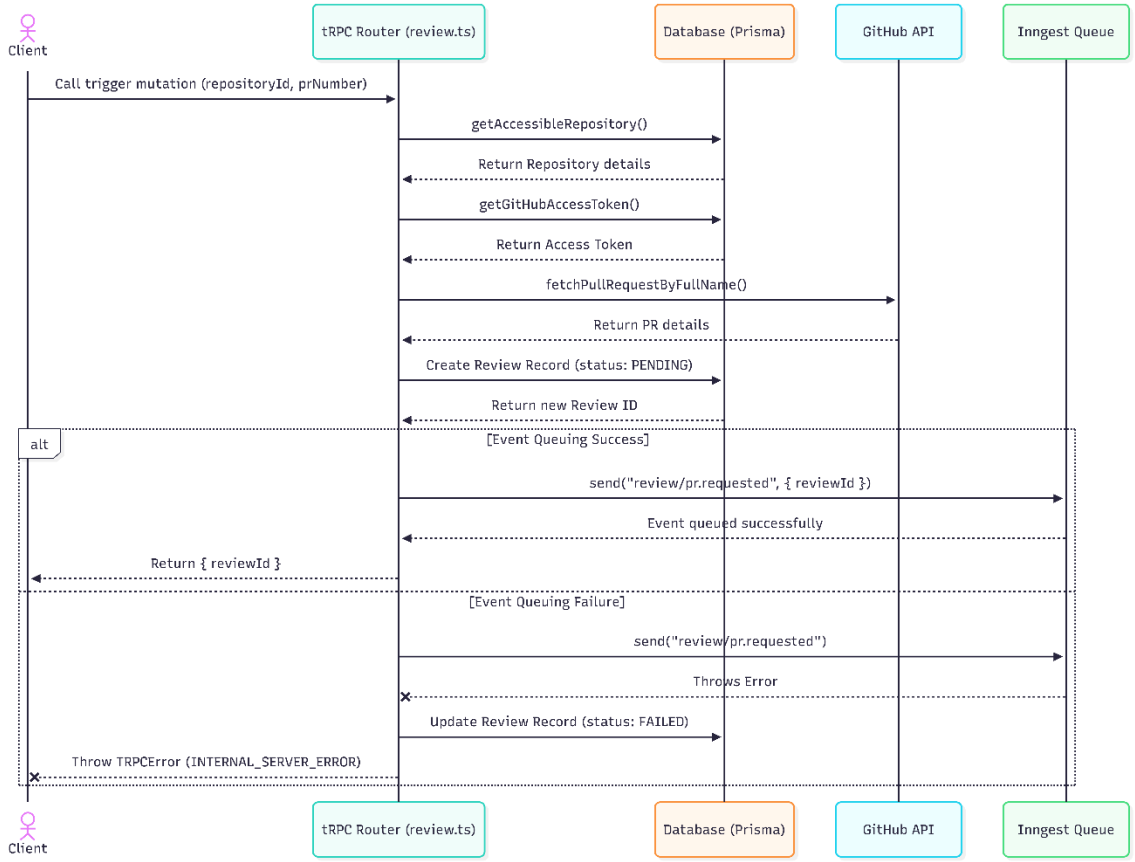


3. Data Flow & System Behavior

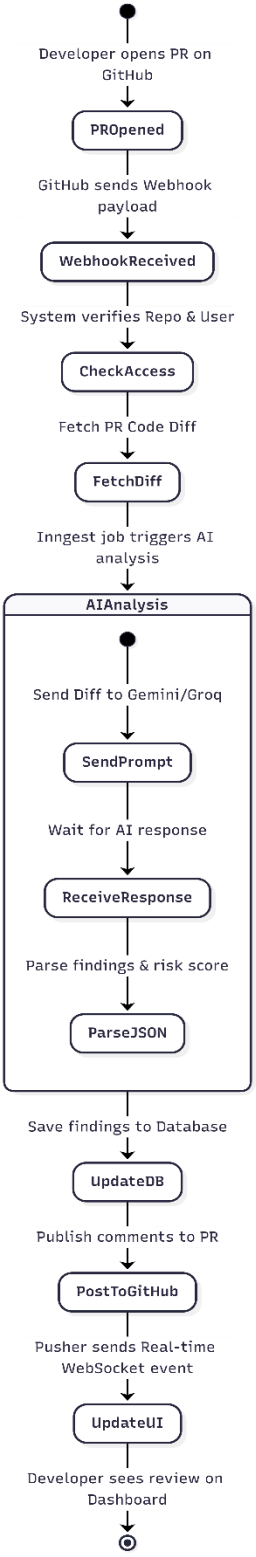
3.1 DFD (Data Flow Diagram)



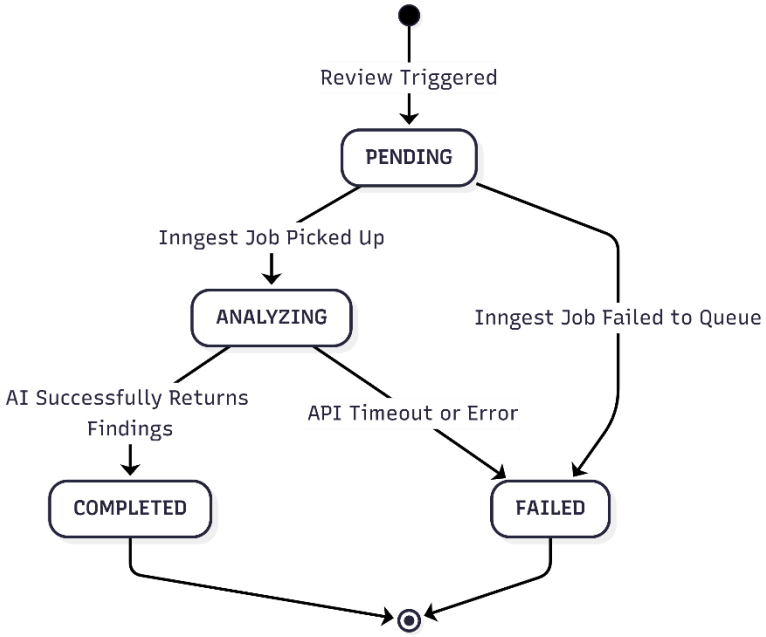
3.2 Sequence Diagram



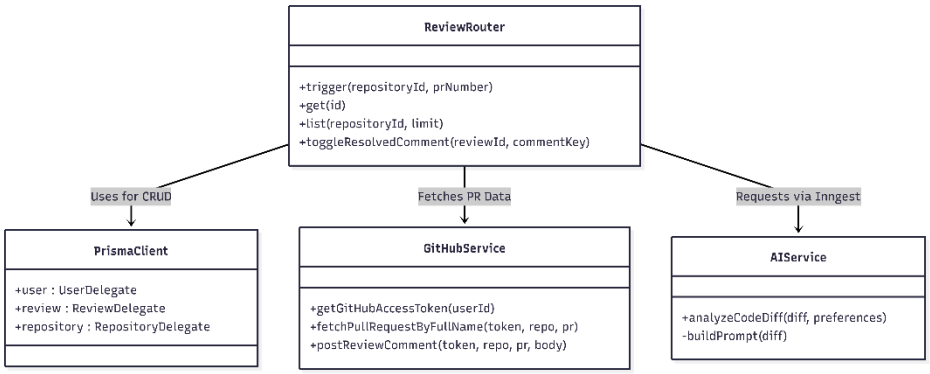
3.3 Activity Diagram



3.4 State Diagram



3.5 Class Diagram



4.2 UI/UX Guidelines

- **Design System:** The application strictly adheres to the **shadcn/ui** component library, ensuring accessible, high-quality Radix UI primitives.
- **Color Palette & Theme:** The system defaults to **Dark Mode** to reduce developer eye strain, utilizing neutral grays (zinc/slate) with semantic colors for status indicators (Red for Critical bugs, Yellow for Warnings, Green for Passed checks).
- **Typography:** Uses the **Geist Sans** and **Geist Mono** font families, optimized for legibility in dense technical dashboards and code snippets.
- **Animations: GSAP** (GreenSock Animation Platform) and **Lenis** (for smooth scrolling) are utilized sparingly to provide fluid page transitions and micro-interactions without impacting overall performance.
- **Responsiveness:** Built with **Tailwind CSS**, ensuring the dashboard is fully responsive across desktop, tablet, and mobile breakpoints, though optimized primarily for desktop (developer environments).

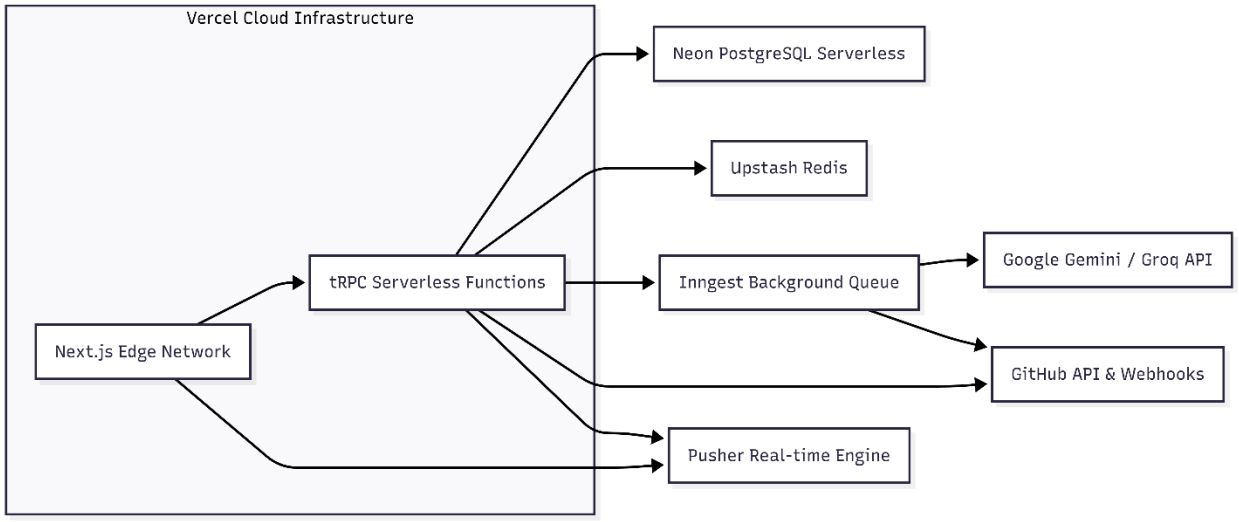
---

5. System Deployment & Integration

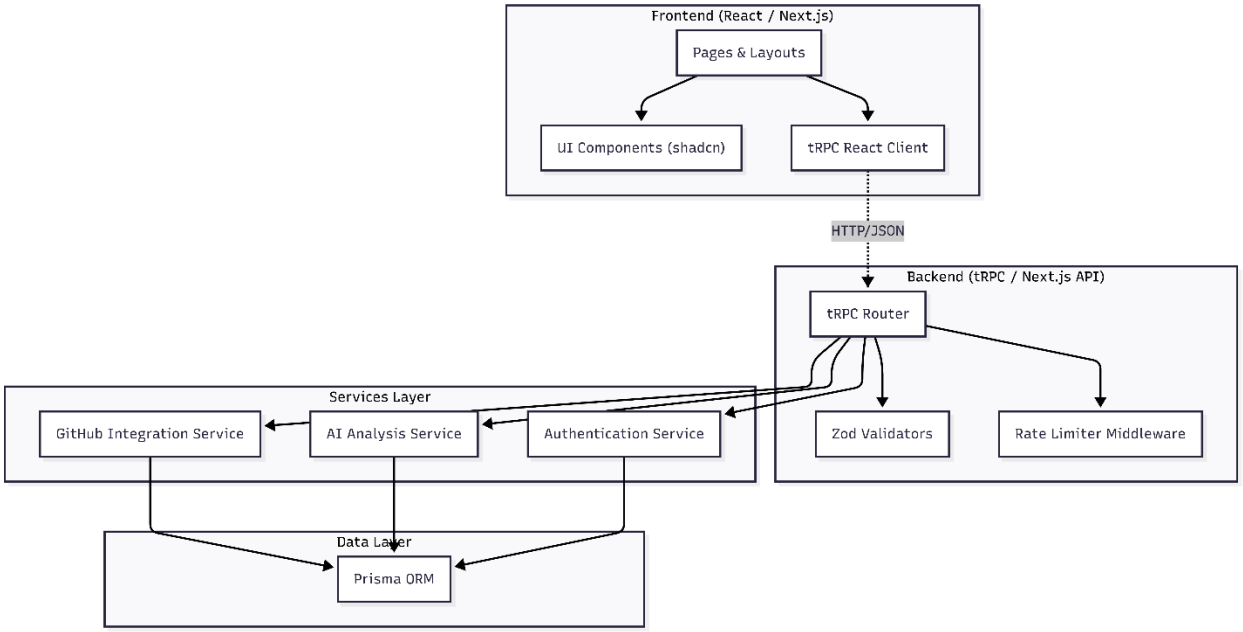
5.1 Technology Stack

- **Frontend:** React 19, Next.js 16 (App Router), Tailwind CSS, shadcn/ui.
- **Backend:** Next.js Serverless Functions, tRPC v11 for API routing.
- **Database:** PostgreSQL hosted on Neon.tech.
- **ORM:** Prisma Client v6.
- **Background Jobs:** Inngest.
- **Real-time Engine:** Pusher Channels.
- **Rate Limiting / Caching:** Upstash Redis.
- **Authentication:** Better-Auth (OAuth integration with GitHub).

5.2 Deployment Diagram



5.3 Component Diagram



6. Additional Deliverables

6.1 API Documentation

Since the system utilizes **tRPC**, traditional REST documentation (like Swagger/OpenAPI) is inherently replaced by **End-to-End Type Safety**.

- **Self-Documenting Code:** Frontend clients automatically infer input requirements and return types from the backend routers defined in src/server/api/routers/.
- **External Webhooks:** The /api/webhooks/github route is thoroughly documented within the repository, defining the expected GitHub event payloads (e.g., pull_request.opened, pull_request.synchronize) and HMAC signature verification methods.

6.2 Testing & Validation Plan

To ensure system stability, the following testing strategies are implemented using **Jest** (configured in package.json):

- **Unit Testing:** Individual utility functions (e.g., the diff-algorithm.ts responsible for parsing code diffs) are tested in isolation to ensure precise string manipulation and finding extractions.
- **Integration Testing:** tRPC callers are tested against a local, isolated test database to verify complete request lifecycles (e.g., ensuring triggerReview successfully creates a DB record and dispatches an Inngest event).

- **User Acceptance Testing (UAT):** Beta testing workflows simulating a complete developer lifecycle: Authoring a PR on GitHub -> Validating Webhook reception -> Checking AI feedback accuracy -> Resolving comments on the DevReview Dashboard.

6.3 Deployment Strategy & Scaling

- **Hosting Environment:** The application is deployed on **Vercel**, leveraging its native support for Next.js App Router, Edge caching, and Serverless Functions.
- **CI/CD Pipeline:** GitHub Actions combined with Vercel's automated deployments ensure that every push to the main branch is built, tested, and deployed securely. Preview environments are automatically generated for feature branches.
- **Scaling Considerations:** * The database utilizes Neon's **Serverless Connection Pooling** to handle massive concurrent database connections generated by serverless API endpoints.
    - Inngest manages heavy background workloads dynamically, preventing API timeouts by placing AI review tasks into a robust, retryable queue system rather than processing them during the HTTP request lifecycle.